



```
In [7]: def heuristic(a, b):
        return abs(a[0] - b[0]) + abs(a[1] - b[1])

def is_valid(grid, x, y):
    return 0 <= x < len(grid) and 0 <= y < len(grid[0]) and grid[x][y] == 0

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    return path[::-1]

def search(grid, start, goal, use_g_score=True):
    open_list = [(start, 0)]
    came_from = {}
    g_score = {start: 0}

    while open_list:
        # Find node with lowest priority
        min_index = 0
        for i in range(1, len(open_list)):
            if open_list[i][1] < open_list[min_index][1]:
                min_index = i

        current, _ = open_list.pop(min_index)

        # Check goal
        if current == goal:
            return reconstruct_path(came_from, current)

        # Explore neighbors
        for dx, dy in [(-1,0), (0,1), (1,0), (0,-1)]:
            neighbor = (current[0] + dx, current[1] + dy)
            if is_valid(grid, *neighbor):
                g = g_score[current] + 1
                if neighbor not in g_score or g < g_score[neighbor]:
                    g_score[neighbor] = g
                    priority = g + heuristic(neighbor, goal) if use_g_score else g
                    open_list.append((neighbor, priority))
                    came_from[neighbor] = current

    return None

# --- Grid ---
grid = [
    [0,0,0,1,0],
    [1,1,0,1,0],
    [0,0,0,0,0],
    [0,1,1,1,1],
    [0,0,0,0,0]
]

start, goal = (0,0), (4,4)
```

```
# --- Run A* ---
path_astar = search(grid, start, goal, use_g_score=True)
print("A* Path:" if path_astar else "No path found")
if path_astar:
    print(path_astar)

# --- Run GBFS ---
path_gbfs = search(grid, start, goal, use_g_score=False)
print("GBFS Path:" if path_gbfs else "No path found")
if path_gbfs:
    print(path_gbfs)
```

A* Path:

```
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
```

GBFS Path:

```
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
```

Feature	**Greedy Best-First Search (GBFS)**
Evaluation Function	Uses only the **heuristic** `h(n)` (estimate to go)
Goal	Finds a path **fast** , not necessarily the shortest
Optimality	**Not guaranteed** to be optimal.
Speed	Often **faster** , explores promising nodes first.
Memory Use	Can expand fewer nodes if heuristic is good.
Path Quality	Path may be longer or suboptimal.

2 second

In []:

In []:

In [13]: # --- Sudoku Solver using Backtracking ---

```
def is_valid(board, row, col, num):
    #Check if placing 'num' in board[row][col] is valid.
    # Row and column check
    for i in range(9):
        if board[row][i] == num or board[i][col] == num:
            return False

    # 3x3 subgrid check
    box_row = (row // 3) * 3
    box_col = (col // 3) * 3
    for i in range(3):
        for j in range(3):
            if board[box_row + i][box_col + j] == num:
                return False
```

```

    return True
def solve_sudoku(board):
    """Solve the Sudoku puzzle using backtracking."""
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0: # Empty cell found
                for num in range(1, 10): # Try numbers 1-9
                    if is_valid(board, row, col, num):
                        board[row][col] = num # Place number
                        if solve_sudoku(board): # Recurse
                            return True
                        board[row][col] = 0 # Backtrack
                return False # No valid number found
    return True # Solved successfully

# --- Example Sudoku Board ---
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

# --- Solve and Print the Sudoku ---
if solve_sudoku(board):
    print("Solved Sudoku:\n")
    for row in board:
        print(row)
else:
    print("No solution exists.")

```

Solved Sudoku:

```

[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]

```

In []:

You can say:

Sudoku is also a CSP.

Variables: 81 cells

Domains: numbers 1-9

Constraints:

Each row must have all digits 1-9

Each column must have all digits 1-9

Each 3×3 box must have all digits 1-9

The solver fills cells one by one, ensuring that every assignment respects these rules.

```
In [16]: def map_coloring(assignment={}):
    if len(assignment) == len(variables):
        return assignment

    for region in variables:
        if region not in assignment:
            for color in domains[region]:
                if all(assignment.get(n) != color for n in neighbors[region]):
                    assignment[region] = color
                    solution = map_coloring(assignment)
                    if solution:
                        return solution
                    del assignment[region]
            return None

    # Example Map: Australia
    variables = ['A', 'B', 'C', 'D']
    domains = {v: ['Red', 'Green', 'Blue'] for v in variables}
    neighbors = {
        'A': ['B', 'C'],
        'B': ['A', 'D'],
        'C': ['A', 'D'],
        'D': ['B', 'C']
    }

    solution = map_coloring()
    print(solution if solution else "No solution exists.")

{'A': 'Red', 'B': 'Green', 'C': 'Green', 'D': 'Red'}
```

In []: In the Map Coloring Problem, each region on a map must be colored so that no t
Each region **is** a variable, possible colors are its domain, **and** the constraint

3

```
In [23]: sys_db ={
    "john": [["fever", "cough", ], ["flu"]],
    "amar": [["fever", "headache", "muscle_pain"], ["malaria"]],
    "rush": [["fever", "diarrhoea", "body_pain"], ["covid"]]
}

disease_db ={
    "flu": ["fever", "cough", "weekness", "myalgia" ],
    "malaria": ["fever", "headache", "muscle_pain", "sweating"],
    "covid": ["fever", "diarrhoea", "body_pain"]
}

def HasSymptom(person, symptom):
    return symptom in sys_db.get(person, [[],[]])[0]
def HasDisease(person, disease):
    return disease in disease_db.get(person, [[],[]])[1]
```

```
In [24]: print(HasDisease("aman","malaria"))
```

False

```
In [25]: print(HasSymptom("john", "headache"))
```

False

```
In [26]: print(HasSymptom("john", "cough"))
```

True

4

```
In [27]: # --- Input patient and symptoms ---
name = input("Enter patient name: ").strip()
symptoms = input(f"Enter symptoms for {name} (comma-separated): ").strip()
facts = set(sym.strip().lower() for sym in symptoms.split(',') if sym.strip())
# --- Input rules ---
print("\nEnter rules like: symptom1,symptom2 => disease")
print("Type 'done' when finished.")
rules = []
while True:
    rule = input("Rule: ").strip().lower()
    if rule == 'done':
        break
    if '=>' in rule:
        conds, concl = rule.split('=>')
        rules.append((set(c.strip() for c in conds.split(',')), concl.strip()))
    else:
        print("Invalid rule format!")
# --- Forward Chaining ---
changed = True
```

```

while changed:
    changed = False
    for conds, concl in rules:
        if conds <= facts and concl not in facts:
            facts.add(concl)
            print(f"Inferred: {concl.title()} from {' '.join(conds).title()}")
            changed = True
23
# --- Output final facts ---
print(f"\nFinal facts for {name.title()}:")
for fact in sorted(facts):
    print(f"{name.title()}({fact.title()})")

```

Enter rules like: symptom1,symptom2 => disease
Type 'done' when finished.
Final facts for Done:
Done(Fewer)

In []:

In []:

In []:

5

```

In [57]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df =pd.read_csv("Bengaluru_House_Data.csv")

```

```

In [58]: df = df.dropna()

df =df[["area_type", "size", "price"]]
df = df.dropna()

x =df[["area_type", "size"]]
y =df["price"]

```

```

In [59]: df.isnull().sum()

```

```

Out[59]: area_type    0
size              0
price             0
dtype: int64

```

```

In [60]: import numpy as np
import matplotlib.pyplot as plt

```

```

# --- Data Generation ---
np.random.seed(42)
n_samples = 200
size = np.random.rand(n_samples) * 200
room = np.random.randint(1, 6, n_samples)
X = np.c_[size, room]
y = 50 * size + 10000 * room + 20000 + np.random.randn(n_samples) * 5000
X = (X - np.mean(X, axis=0)) / np.std(X, axis=0)

# --- Helper functions ---
def predict(X, w, b):
    return np.dot(X, w) + b

def compute_cost(X, y, w, b):
    m = len(y)
    y_pred = predict(X, w, b)
    return (1 / (2 * m)) * np.sum((y_pred - y) ** 2)

def compute_gradient(X, y, w, b):
    m = len(y)
    y_pred = predict(X, w, b)
    dw = (1 / m) * np.dot(X.T, (y_pred - y))
    db = (1 / m) * np.sum(y_pred - y)
    return dw, db

# --- Batch Gradient Descent ---
def bgd(X, y, lr=0.01, epochs=10):
    m, n = X.shape
    w = np.zeros(n)
    b = 0
    cost_history = []
    for epoch in range(epochs):
        dw, db = compute_gradient(X, y, w, b)
        w -= lr * dw
        b -= lr * db
        cost = compute_cost(X, y, w, b)
        cost_history.append(cost)
        print(f"[BGD] Epoch {epoch+1}: Cost={cost:.2f}")
    return w, b, cost_history

# --- Stochastic Gradient Descent ---
def sgd(X, y, lr=0.01, epochs=10):
    m, n = X.shape
    w = np.zeros(n)
    b = 0
    cost_history = []
    for epoch in range(epochs):
        for i in range(m):
            rand_i = np.random.randint(0, m)
            xi = X[rand_i:rand_i + 1]
            yi = y[rand_i:rand_i + 1]
            dw, db = compute_gradient(xi, yi, w, b)
            w -= lr * dw

```

```

        b -= lr * db
        cost = compute_cost(X, y, w, b)
        cost_history.append(cost)
        print(f"[SGD] Epoch {epoch+1}: Cost={cost:.2f}")
    return w, b, cost_history

# --- Mini-Batch Gradient Descent ---
def mbg(X, y, lr=0.01, epochs=10, batch_size=32):
    m, n = X.shape
    w = np.zeros(n)
    b = 0
    cost_history = []
    for epoch in range(epochs):
        indices = np.random.permutation(m)
        X_shuffled, y_shuffled = X[indices], y[indices]
        for i in range(0, m, batch_size):
            x_batch = X_shuffled[i:i + batch_size]
            y_batch = y_shuffled[i:i + batch_size]
            dw, db = compute_gradient(x_batch, y_batch, w, b)
            w -= lr * dw
            b -= lr * db
        cost = compute_cost(X, y, w, b)
        cost_history.append(cost)
        print(f"[MBGD] Epoch {epoch+1}: Cost={cost:.2f}")
    return w, b, cost_history

# --- Train all models ---
w_bgd, b_bgd, cost_bgd = bgd(X, y, lr=0.01, epochs=50)
w_sgd, b_sgd, cost_sgd = sgd(X, y, lr=0.01, epochs=50)
w_mbgd, b_mbgd, cost_mbgd = mbg(X, y, lr=0.01, epochs=50, batch_size=32)

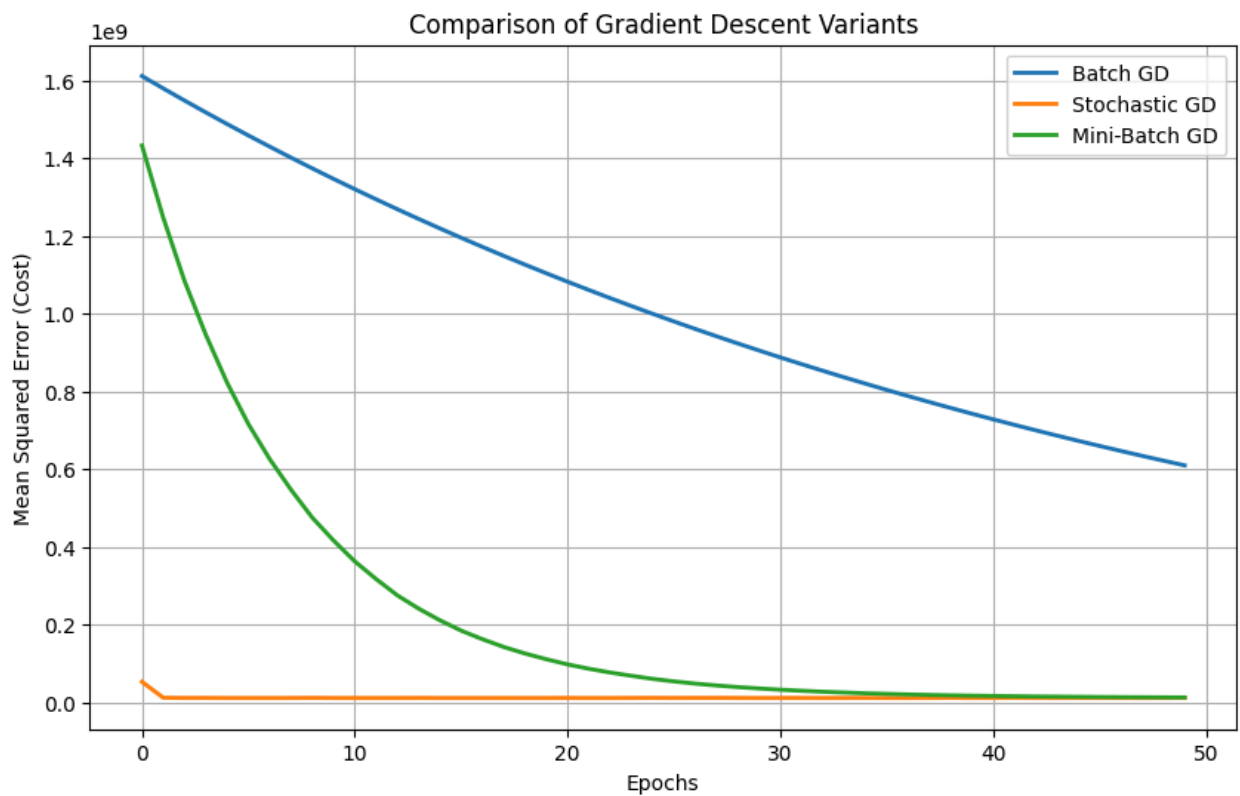
# --- Plot Cost Curves ---
plt.figure(figsize=(10, 6))
plt.plot(cost_bgd, label="Batch GD", linewidth=2)
plt.plot(cost_sgd, label="Stochastic GD", linewidth=2)
plt.plot(cost_mbgd, label="Mini-Batch GD", linewidth=2)
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error (Cost)")
plt.title("Comparison of Gradient Descent Variants")
plt.legend()
plt.grid(True)
plt.show()

```


[BGD] Epoch 1: Cost=1611000197.47
[BGD] Epoch 2: Cost=1579203031.84
[BGD] Epoch 3: Cost=1548038199.95
[BGD] Epoch 4: Cost=1517493119.47
[BGD] Epoch 5: Cost=1487555458.58
[BGD] Epoch 6: Cost=1458213130.99
[BGD] Epoch 7: Cost=1429454291.03
[BGD] Epoch 8: Cost=1401267328.89
[BGD] Epoch 9: Cost=1373640865.87
[BGD] Epoch 10: Cost=1346563749.82
[BGD] Epoch 11: Cost=1320025050.61
[BGD] Epoch 12: Cost=1294014055.72
[BGD] Epoch 13: Cost=1268520265.87
[BGD] Epoch 14: Cost=1243533390.82
[BGD] Epoch 15: Cost=1219043345.18
[BGD] Epoch 16: Cost=1195040244.32
[BGD] Epoch 17: Cost=1171514400.40
[BGD] Epoch 18: Cost=1148456318.42
[BGD] Epoch 19: Cost=1125856692.39
[BGD] Epoch 20: Cost=1103706401.60
[BGD] Epoch 21: Cost=1081996506.85
[BGD] Epoch 22: Cost=1060718246.93
[BGD] Epoch 23: Cost=1039863034.99
[BGD] Epoch 24: Cost=1019422455.12
[BGD] Epoch 25: Cost=999388258.93
[BGD] Epoch 26: Cost=979752362.22
[BGD] Epoch 27: Cost=960506841.70
[BGD] Epoch 28: Cost=941643931.78
[BGD] Epoch 29: Cost=923156021.44
[BGD] Epoch 30: Cost=905035651.15
[BGD] Epoch 31: Cost=887275509.87
[BGD] Epoch 32: Cost=869868432.06
[BGD] Epoch 33: Cost=852807394.79
[BGD] Epoch 34: Cost=836085514.92
[BGD] Epoch 35: Cost=819696046.30
[BGD] Epoch 36: Cost=803632377.05
[BGD] Epoch 37: Cost=787888026.88
[BGD] Epoch 38: Cost=772456644.47
[BGD] Epoch 39: Cost=757332004.90
[BGD] Epoch 40: Cost=742508007.14
[BGD] Epoch 41: Cost=727978671.58
[BGD] Epoch 42: Cost=713738137.61
[BGD] Epoch 43: Cost=699780661.24
[BGD] Epoch 44: Cost=686100612.82
[BGD] Epoch 45: Cost=672692474.70
[BGD] Epoch 46: Cost=659550839.05
[BGD] Epoch 47: Cost=646670405.65
[BGD] Epoch 48: Cost=634045979.76
[BGD] Epoch 49: Cost=621672470.00
[BGD] Epoch 50: Cost=609544886.31
[SGD] Epoch 1: Cost=52828460.41
[SGD] Epoch 2: Cost=11872899.39
[SGD] Epoch 3: Cost=11314999.03
[SGD] Epoch 4: Cost=11418956.96

[SGD] Epoch 5: Cost=11175031.72
[SGD] Epoch 6: Cost=11137335.44
[SGD] Epoch 7: Cost=11120060.70
[SGD] Epoch 8: Cost=11248620.40
[SGD] Epoch 9: Cost=11558475.20
[SGD] Epoch 10: Cost=11207259.01
[SGD] Epoch 11: Cost=11186086.90
[SGD] Epoch 12: Cost=11123337.73
[SGD] Epoch 13: Cost=11208075.03
[SGD] Epoch 14: Cost=11360082.94
[SGD] Epoch 15: Cost=11160083.92
[SGD] Epoch 16: Cost=11138381.87
[SGD] Epoch 17: Cost=11158377.33
[SGD] Epoch 18: Cost=11184470.04
[SGD] Epoch 19: Cost=11136895.65
[SGD] Epoch 20: Cost=11195865.44
[SGD] Epoch 21: Cost=11347364.40
[SGD] Epoch 22: Cost=11118292.76
[SGD] Epoch 23: Cost=11175876.62
[SGD] Epoch 24: Cost=11321688.53
[SGD] Epoch 25: Cost=11558735.97
[SGD] Epoch 26: Cost=11307911.76
[SGD] Epoch 27: Cost=11458822.19
[SGD] Epoch 28: Cost=11476519.54
[SGD] Epoch 29: Cost=11289329.09
[SGD] Epoch 30: Cost=11155212.98
[SGD] Epoch 31: Cost=11262412.99
[SGD] Epoch 32: Cost=11320376.83
[SGD] Epoch 33: Cost=11156501.78
[SGD] Epoch 34: Cost=11288973.77
[SGD] Epoch 35: Cost=11400287.41
[SGD] Epoch 36: Cost=11187880.06
[SGD] Epoch 37: Cost=11165622.93
[SGD] Epoch 38: Cost=11136899.13
[SGD] Epoch 39: Cost=11528212.07
[SGD] Epoch 40: Cost=11109212.86
[SGD] Epoch 41: Cost=11248524.87
[SGD] Epoch 42: Cost=11505884.93
[SGD] Epoch 43: Cost=11208518.88
[SGD] Epoch 44: Cost=11178815.68
[SGD] Epoch 45: Cost=11263661.65
[SGD] Epoch 46: Cost=11205497.42
[SGD] Epoch 47: Cost=11308933.25
[SGD] Epoch 48: Cost=11165260.30
[SGD] Epoch 49: Cost=11352469.79
[SGD] Epoch 50: Cost=11420381.37
[MBGD] Epoch 1: Cost=1432273249.84
[MBGD] Epoch 2: Cost=1246503118.25
[MBGD] Epoch 3: Cost=1083693761.97
[MBGD] Epoch 4: Cost=945367732.96
[MBGD] Epoch 5: Cost=822125342.00
[MBGD] Epoch 6: Cost=715809579.22
[MBGD] Epoch 7: Cost=625897299.11
[MBGD] Epoch 8: Cost=547117574.24

[MBGD] Epoch 9: Cost=475343499.13
[MBGD] Epoch 10: Cost=416359260.86
[MBGD] Epoch 11: Cost=362572593.86
[MBGD] Epoch 12: Cost=317238978.17
[MBGD] Epoch 13: Cost=275142888.95
[MBGD] Epoch 14: Cost=241104579.79
[MBGD] Epoch 15: Cost=211051464.00
[MBGD] Epoch 16: Cost=184418107.86
[MBGD] Epoch 17: Cost=162363806.69
[MBGD] Epoch 18: Cost=142397420.34
[MBGD] Epoch 19: Cost=125555880.85
[MBGD] Epoch 20: Cost=110865292.47
[MBGD] Epoch 21: Cost=97539313.43
[MBGD] Epoch 22: Cost=86390565.31
[MBGD] Epoch 23: Cost=76688539.76
[MBGD] Epoch 24: Cost=68208734.77
[MBGD] Epoch 25: Cost=60277675.69
[MBGD] Epoch 26: Cost=53835060.21
[MBGD] Epoch 27: Cost=48133767.38
[MBGD] Epoch 28: Cost=43343882.49
[MBGD] Epoch 29: Cost=39166226.96
[MBGD] Epoch 30: Cost=35644809.66
[MBGD] Epoch 31: Cost=32471303.42
[MBGD] Epoch 32: Cost=29728667.48
[MBGD] Epoch 33: Cost=27222332.11
[MBGD] Epoch 34: Cost=25196016.87
[MBGD] Epoch 35: Cost=23171077.65
[MBGD] Epoch 36: Cost=21647640.79
[MBGD] Epoch 37: Cost=20190054.12
[MBGD] Epoch 38: Cost=19020054.07
[MBGD] Epoch 39: Cost=17977085.95
[MBGD] Epoch 40: Cost=17071227.39
[MBGD] Epoch 41: Cost=16302482.53
[MBGD] Epoch 42: Cost=15686160.39
[MBGD] Epoch 43: Cost=15052564.74
[MBGD] Epoch 44: Cost=14557229.03
[MBGD] Epoch 45: Cost=14075868.18
[MBGD] Epoch 46: Cost=13672958.72
[MBGD] Epoch 47: Cost=13308351.97
[MBGD] Epoch 48: Cost=13022322.70
[MBGD] Epoch 49: Cost=12792708.32
[MBGD] Epoch 50: Cost=12550810.54



In [61]:

Type	Data Used per Update	Convergence Speed	Stability
Batch GD	All samples	Slow	Very stable
Stochastic GD	1 sample	Fast (per update)	Noisy
Mini-Batch GD	Few samples (e.g., 32-128)	Fast + Stable	Balance

```
Cell In[61], line 5
| Mini-Batch GD | Few samples (e.g., 32-128) | Fast + Stable | Balanced
| Deep Learning |
SyntaxError: invalid character '-' (U+2013)
```

8

In []:

```
import numpy as np
import matplotlib.pyplot as plt

class WTA:
    def __init__(self, num_neurons, input_dim, learning_rate=0.1):
        self.num_neurons = num_neurons
        self.input_dim = input_dim
        self.learning_rate = learning_rate
        self.weights = np.random.rand(num_neurons, input_dim)

    def find_winner(self, x):
        return np.argmax(np.linalg.norm(self.weights - x, axis=1))
```

```

def train(self, data, epochs=50):
    for _ in range(epochs):
        for x in data:
            w = self.find_winner(x)
            self.weights[w] += self.learning_rate * (x - self.weights[w])

# Generate 2D clustered data
np.random.seed(42)
data = np.vstack([
    np.random.randn(100, 2) * 0.3 + [1, 1],
    np.random.randn(100, 2) * 0.3 + [3, 3],
    np.random.randn(100, 2) * 0.3 + [5, 1]
])

# Train WTA network
wta = WTA(num_neurons=3, input_dim=2, learning_rate=0.1)
wta.train(data, epochs=25)

# Assign and print clusters
labels = np.array([wta.find_winner(x) for x in data])
for i, x in enumerate(data[:10]): # print first 10 points only
    print(f"Point {i+1}: {x} → Cluster {labels[i+1]}")

# Plot results
colors = ['r', 'g', 'b']
plt.figure(figsize=(8,6))
for i in range(3):
    plt.scatter(data[labels==i,0], data[labels==i,1], c=colors[i], label=f'Cluster {i+1}')
plt.scatter(wta.weights[:,0], wta.weights[:,1], c='k', marker='X', s=200, label='Weights')
plt.title("Winner-Takes-All (WTA) Clustering")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)
plt.show()

```

10

```

In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
class SOM:
    def __init__(self, k, lr=0.5):
        self.k = k
        self.lr = lr
        self.weights = np.random.randn(k, 3)
    def find_winner(self, x):
        return np.argmin(np.linalg.norm(self.weights - x, axis = 1))
    def train(self, data, epochs = 10):
        for _ in range(epochs):
            for x in data:

```

```

        w = self.find_winner(x)
        self.weights[w] += self.lr * (x - self.weights[w])
        self.lr *= 0.9
img = Image.open(r"C:\Users\Admin\Downloads\download.jpeg").resize((200,200))
data = np.array(img) / 255.0
pixels = data.reshape(-1,3)

som = SOM(k = 16,lr = 0.5)
som.train(pixels, epochs = 10)

comp = np.array([som.weights[som.find_winner(p)] for p in pixels ])
comp_img = comp.reshape(data.shape)
plt.subplot(1,2,1);
plt.imshow(data);
plt.title('Original');
plt.axis('off')

plt.subplot(1,2,2);
plt.imshow(comp_img);
plt.title('compressed');
plt.axis('off')

plt.show()

```